

Statistische Programmierung: Lösungsheft

Hochschule Kempten

Linus Konstanzer

2026-10-01

Inhaltsverzeichnis

	3
1 Einführung in R: Lösungen	4
1.1 Lösungen	4

Dieses Dokument enthält die Lösungen der Übungsaufgaben.

1 Einführung in R: Lösungen

1.1 Lösungen

Solution 1.1.

```
2 + 3 * 4
```

```
[1] 14
```

Mit diesen Befehlen wird die Rechnung $2 + 3 * 4$ durchgeführt.

```
(2 + 3) * 4
```

```
[1] 20
```

Im Vergleich zur vorherigen Aufgabe werden hier Klammern gesetzt, um zuerst $2 + 3$ zu berechnen, bevor das Ergebnis mit 4 multipliziert wird.

```
0.2 * 3 - 1.1
```

```
[1] -0.5
```

Neben ganzzahligen Werten, kann R auch Rechnungen mit Dezimalzahlen durchführen.

```
0,2 * 3 - 1.1
```

```
Error in parse(text = input): <text>:1:2: unexpected '','  
1: 0,  
   ^
```

Bei der obigen Rechnung wird ein Fehler ausgegeben, da das Dezimaltrennzeichen in R nicht ,, sondern . ist.

```
2^3^2
```

```
[1] 512
```

Die obige Code Zeile führt die Rechnung $2^{3^2} = 2^9$ aus.

```
(2^3)^2
```

```
[1] 64
```

Wie zuvor wird durch Klammern bewirkt, dass die erste Potenzierung zuerst durchgeführt wird.

```
exp(1)
```

```
[1] 2.718282
```

Die `exp(x)` Funktion führt die Rechnung e^x durch, wobei $e \approx 2.718282$ die eulersche Konstante ist.

```
log(exp(1))
```

```
[1] 1
```

Die `log` Funktion berechnet den natürlichen Logarithmus. Die Rechnung $\log \circ \exp$ ist somit die Identitätsfunktion, also $x \mapsto x$.

```
sqrt(16)
```

```
[1] 4
```

Die `sqrt(x)` Funktion führt die Rechnung $\sqrt{x}$ durch.

```
16^(1/2)
```

```
[1] 4
```

Da $\sqrt{16} = 16^{0.5} = 16^{\frac{1}{2}}$ ist das Ergebnis identisch zur vorherigen Code Zelle.

```
Sqrt(16)
```

```
Error in `Sqrt()` :  
! could not find function "Sqrt"
```

Hier wird ein Fehler ausgegeben, da R *Case Sensitive* ist. Case Sensitivity führt dazu, dass bei dem Eingeben von Funktionen auch auf Groß- und Kleinschreibung geachtet werden muss.

```
# Das ist ein Kommentar.
```

Kommentare in R werden durch das # Symbol initiiert. Kommentare werden nicht ausgewertet, so dass hinter dem # Symbol jegliche Folge von Zeichen stehen kann.

Solution 1.2.

```
x <- 4  
x2 <- sqrt(3*x^2+log(1/exp(x))+5)
```

```
x
```

```
[1] 4
```

```
x2
```

```
[1] 7
```

```
x + x2
```

```
[1] 11
```

```
x.Produkt <- x * x2  
x.Produkt
```

```
[1] 28
```

```
x.Produkt <- x.Produkt * x  
ls()
```

```
[1] "data_rahmen"          "data_rahmen_filtered" "x"  
[4] "x.Produkt"           "x2"
```

```
rm(x)
```

Durch die `rm` Funktion wird die Variable `x` entfernt und kann somit nicht mehr aufgerufen werden. Deshalb wird beim versuchten Aufruf in der folgenden Zeile eine Fehlermeldung ausgegeben.

```
x
```

```
Error:
! object 'x' not found
```

Solution 1.3. Die Variable `keine.Zahl` ist vom Datentyp `<char>` und kann somit nicht ohne weiteres wie ein tatsächlicher Zahlenwert gehandhabt werden. Das führt dazu, dass in der folgende Code Zelle eine Fehlermeldung ausgegeben wird.

```
Zahl <- 10
keine.Zahl <- "10"
Zahl + 1
```

```
[1] 11
```

```
keine.Zahl + 1
```

```
Error in `keine.Zahl + 1`:
! non-numeric argument to binary operator
```

Solution 1.4.

```
5:9
```

```
[1] 5 6 7 8 9
```

```
10:1
```

```
[1] 10 9 8 7 6 5 4 3 2 1
```

```
seq(from=-0.1, to=0, by=0.02)
```

```
[1] -0.10 -0.08 -0.06 -0.04 -0.02  0.00
```

```
seq(from=10000, to=20000, length.out=5)
```

```
[1] 10000 12500 15000 17500 20000
```

```
rep(-3:2, 3)
```

```
[1] -3 -2 -1  0  1  2 -3 -2 -1  0  1  2 -3 -2 -1  0  1  2
```

```
c(5:10, seq(from=10.1, by=.1, to=10.5))
```

```
[1]  5.0  6.0  7.0  8.0  9.0 10.0 10.1 10.2 10.3 10.4 10.5
```

Solution 1.5. Gegeben sind die Vektoren:

```
x <- 4:2  
y <- seq(from=0.1, to=0.5, by=0.1)
```

Da die Objekte `x` und `y` nicht die selbe Länge besitzen, werden in der folgenden Zelle die Einträge des kürzeren Vektors bis zur Länge des längeren Vektors wiederholt. Anschließend werden die einzelnen Einträge komponentenweise addiert.

```
x + y
```

```
Warning in x + y: longer object length is not a multiple of shorter object length
```

```
[1] 4.1 3.2 2.3 4.4 3.5
```

In der folgenden Code Zelle werden die Listeneinträge von `x` analog erweitert und dann komponentenweise mit den Einträgen von `y` multipliziert.


```
x * y
```

Warning in `x * y`: longer object length is not a multiple of shorter object length

```
[1] 0.4 0.6 0.6 1.6 1.5
```

Durch den `^` Operator werden zuerst alle Elemente der Liste `x` potenziert und anschließend mit dem Wert 1 addiert.

```
x^3 + 1
```

```
[1] 65 28 9
```

Analog wie in den vorherigen Teilaufgaben werden die konstanten Faktoren zuerst komponentenweise multipliziert, bevor die Liste `x` erweitert werden und von `3*y` komponentenweise abgezogen werden.

```
2*x - 3*y
```

Warning in `2 * x - 3 * y`: longer object length is not a multiple of shorter object length

```
[1] 7.7 5.4 3.1 6.8 4.5
```

Die Länge der Liste `x+y` ist gegeben durch die Länge des längsten Elements in der Summe. Also durch die Länge von `y`.

```
n <- length(x + y)
```

Warning in `x + y`: longer object length is not a multiple of shorter object length

Analog wie in den vorherigen Teilaufgaben wird die Summe von `x` und `y` berechnet, dann die Summe der einzelnen Komponenten und zum Schluss wird die Summe durch `n` geteilt.

```
sum(x+y)/n
```

Warning in x + y: longer object length is not a multiple of shorter object length

```
[1] 3.5
```

Solution 1.6.

```
x <- seq(from=0, to=100, by=2)
y <- 100:1
x[3]
```

```
[1] 4
```

```
y[c(1,3,10)]
```

```
[1] 100 98 91
```

```
x[1:4]
```

```
[1] 0 2 4 6
```

```
x[x > 91]
```

```
[1] 92 94 96 98 100
```

```
x[x > 20 & x <= 30]
```

```
[1] 22 24 26 28 30
```

```
y[y==5 | y>95 | y<3]
```

```
[1] 100 99 98 97 96 5 2 1
```

Solution 1.7.

```
data_ramen %>%
  group_by(country) %>%
  summarise(n=n()) %>%
  arrange(desc(n)) %>%
  head(5)
```

```
# A tibble: 5 x 2
  country      n
  <chr>      <int>
1 Japan      352
2 USA        323
3 South Korea 307
4 Taiwan     224
5 Thailand   191
```

Solution 1.8.

```
data_ramen %>%
  filter(country=="Germany") %>%
  nrow()
```

```
[1] 27
```

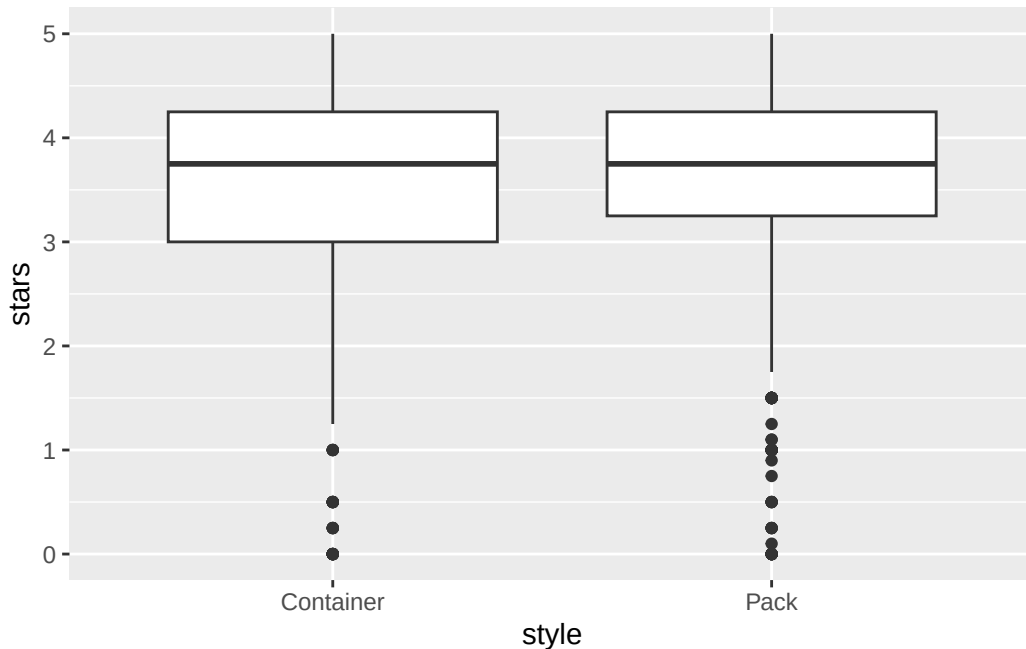
Solution 1.9.

```
1 data_ramen_filtered <- data_ramen %>%
2   filter(style %in% c("Bowl", "Cup", "Pack")) %>%
3   mutate(style = if_else(
4     style=="Pack", "Pack", "Container"
5   ),
6   style= factor(style)
7 )
```

1. In der ersten Zeile wird ein neuer Datensatz `data_ramen_filtered` erzeugt indem der Datensatz `data_ramen` modifiziert wird.
2. In der zweiten Zeile wird die `filter` Funktion auf den Datensatz `data_ramen` angewandt. Der `%in%` Operator prüft hierbei, ob in der Spalte `style` die Werte "Bowl", "Cup", "Pack" vorhanden sind und geben den Wert TRUE oder FALSE zurück. D.h., es werden alle Einträge selektiert, welche den Style "Bowl", "Cup", oder "Pack" enthalten.
3. In der dritten Zeile wird die `mutate` Funktion auf die Variable `style` angewendet.

4. Die Transformation erfolgt hierbei in der vierten und fünften Zeile durch die `if_else` Funktion, welche die Kondition `style=="Pack"` überprüft. Falls der Style eines Eintrags "Pack" ist, so wird diesem wieder der Wert "Pack" zugeschrieben. Falls allerdings der Style "Cup" oder "Bowl" beträgt, so wird der Wert mit "Container" überschrieben.
5. In der sechsten Zeile wird die Variable "Style" dann in eine Faktorvariable umgewandelt.

Solution 1.10. Beschreibe die folgende Grafik. Vergleiche dabei auch die beiden Gruppen untereinander.



Die Grafik zeigt zwei Boxplots für die Variable `star`. Auf der linken Seite wird ein Boxplot für alle Einträge welche den Style "Container" besitzen angezeigt und auf der rechten Seite ein Boxplot für alle Einträge welche den Style "Pack" besitzen. Es lässt sich an den Boxplots ablesen, dass die beiden Styles einen ähnlichen Median bezüglich der Bewertung besitzen. Allerdings ist das 25% Quartil beim Style "Container" etwas kleiner als beim Style "Pack". Beim Style "Pack" werden auf der anderen Seite aber mehr Ausreißer nach Unten abgebildet.

Solution 1.11. Verwende den Datensatz `data_ramen_filtered` um die Dichte der Variable `stars` mit `{ggplot2}` darzustellen. *Hinweis: Die Dichte einer Variable kann mithilfe der Funktion `geom_density` dargestellt werden.*

```
data_ramen_filtered %>%
  ggplot(aes(x=stars))+
```

```
geom_density(fill="#69b3a2", color="#69b3a2")+  
theme_minimal()
```

